

Домашнее задание 6

Flow поверх проекта из ДЗ №4/5

Берёте **готовый проект ДЗ №4** (открытый API + список/детали + ViewModel/Repo/coroutines + Hilt + Room) и добавляете:

- **Flow / StateFlow / SharedFlow**
 - **осмысленную реактивную логику**, где Flow реально решает задачу, а не просто заменяет обычный стейт
-

Обязательные требования

1) Flow используется по назначению

В работе должно быть **минимум одно осмысленное реактивное поведение**, где состояние экрана собирается **из нескольких потоков**.

Подходящие примеры:

- поиск по мере ввода: `query -> debounce -> distinctUntilChanged -> flatMapLatest(search)`
- объединение **поиска + фильтра + данных из Room**
- отображение списка, зависящего от **настроек пользователя + локальной БД + запроса**
- ручной refresh/retry как поток действий пользователя
- история/избранное/кэш, которые автоматически влияют на UI без ручной перезагрузки
- Придумать свой юзкейс

2) Не принимается flow как простой State

Следующие варианты **не засчитываются как тема ДЗ**:

- просто хранить весь экран в `MutableStateFlow` и делать только `_state.value = _state.value.copy(...)`
- использовать `StateFlow` просто как «замену var»
- использовать `SharedFlow` только как one-shot для snackbar/navigation/toast
- оборачивать одиночный `suspend` -вызов в Flow без нормальной реактивной цепочки и без смысла

3) Должно быть минимум 3 источника данных/событий

Минимум 3 независимых источника, есть осмысленная композиция потоков, операторы выбраны по задаче, Flow влияет на реальное поведение приложения

Например:

- строка поиска
- выбранный фильтр/режим
- данные из Room (Flow)
- настройки из DataStore (Flow)
- поток действий пользователя: refresh/retry/search submit
- любой другой

Хотя бы **один источник должен приходить из data layer**, а не быть создан только внутри UI.

4) Должны использоваться операторы Flow

Нужно показать **не только хранение состояния**, но и **преобразование потоков**.

хотя бы один оператор объединения: combine / merge / zip

хотя бы один оператор преобразования или управления потоком: map, mapLatest, flatMapLatest, debounce, distinctUntilChanged, scan

stateIn и catch сами по себе не считаются достаточными

5) StateFlow и SharedFlow должны быть применены уместно

- StateFlow - для **длительного состояния**, у которого есть актуальное текущее значение
- SharedFlow - для **потока действий/триггеров**, если это действительно поток событий, а не просто «сигнал на один раз»

6) Экран должен оставаться рабочим

Проект из ДЗ №4 остаётся рабочим:

- 2+ экрана: **List/Search** и **Detail/{id}**
- Navigation Compose
- Hilt
- Room
- Retrofit + coroutines + viewModelScope
- UI-состояния: **Loading / Error + Retry / Empty / Success**

Что сдавать

1. PR на рабочий проект
-

Минимальный чек-лист перед сдачей

Работа не принимается, если:

- нет реального объединения потоков
 - весь Flow сводится к `MutableStateFlow + copy`
 - `SharedFlow` используется только как one-shot event bus
 - логика Flow есть только формально, но приложение по факту работает так же, как без Flow
 - экран собирается менее чем из **3 независимых источников**
 - Flow используется только в одном месте, а остальная логика остаётся полностью императивной
 - единственный “реактивный” источник - это `Flow` из Room без осмысленной композиции с другими потоками
-

Дедлайн и просрочка

Отсчёт дедлайна начинается с **24.03.2026**. Каждую неделю просрочки по минус баллу, максимум до **-4 баллов**.

Критерии оценки

1) Работоспособность (0–5)

- 0 — не собирается / не запускается
- 3 — основной сценарий работает, но есть баги
- 5 — всё заявленное работает, нет критичных багов

2) Использование Flow (0–4)

- 0 — тема Flow по сути не раскрыта
- 2 — Flow есть, но применение местами формальное

- 4 — Flow действительно уместен: есть композиция потоков, операторы выбраны осознанно, архитектура не выглядит искусственной

3) Субъективный балл (-1..1)

Принято: итог ≥ 6

Просрочка дедлайна: -1 балл каждую неделю